MoSec Mandatory II

Hans Toftegaard
Kenneth Hermann
Mads Lindersson

Sikkerhedsstrategi

Der blev som det første undersøgt og dokumenteret de forskellige IP-adresser, gateways og subnets der eksistere i MoSec.

Host	**IPv4/CIDR**	**Gateway**	**Lokalt netværk/subnet**
Admin	10.10.0.10/26	10.10.0.1	10.10.0.0/26
Employee	10.10.0.20/26	10.10.0.1	10.10.0.0/26
Server	10.10.0.70/26	10.10.0.65	10.10.0.64/26
Guest	10.10.0.140/26	10.10.0.129	10.10.0.128/26
Printer	10.10.0.200/26	10.10.0.193	10.10.0.192/26

Der er observeret at Admin og Employee begge er på samme subnet, men med forskellige business requirements, dette skulle der tages højde for da der skulle laves ACL'er.

Herefter blev der opsat ACL regler ud fra de leverede requirements fra MoSec. Der blev overvejet hvordan subnettet med Admin og Employee skulle administreres. Der var forskellige måder at opsætte ACL'er på dette subnet. Der kunne laves ACL'er for de 2 enkelte IP-adresser, eller der kunne opsættes generelle ACL'er for subnettet og individuelle ACL'er for hver adresse hvis nødvendigt. Der blev besluttet at lave generelle ACL'er for subnettet, og en individuel ACL for Employee, da Employees ikke må kommunikere med gæster.

Grunden til dette valg er for at undgå at lave en masse ACL'er som ender med at blive uoverskueligt, samt at det bliver sværere at vedligeholde hvis der ansættes flere medarbejdere. Ved at opsætte generelle ACL'er for admin/employee subnettet skal der kun opsættes nye ACL'er hvis der ansættes flere medarbejdere.

Færdig ACL tabel

Effective Rule Order

Rules are evaluated from top to bottom. The first matching rule decides the action.

Priority	Type	Incoming Zone	Source	Destination	Action	Description
1	Firewall	Guest	10.10.0.0/26	0.0.0.0/0	DENY	guest anti-spoof
2	ACL	Any	10.10.0.128/26	10.10.0.0/26	DENY	guest to admin/employee
3	ACL	Any	10.10.0.128/26	10.10.0.64/26	DENY	guest to server
4	ACL	Any	10.10.0.128/26	10.10.0.192/26	DENY	guest to printer
5	ACL	Any	10.10.0.128/26	0.0.0.0/0	ALLOW	guest to internet
6	ACL	Any	10.10.0.20	10.10.0.128/26	DENY	employee to guest
7	ACL	Any	10.10.0.0/26	0.0.0.0/0	ALLOW	admin + employee to all
8	Firewall	Guest	10.10.0.64/26	0.0.0.0/0	DENY	guest anti-spoof server
9	Firewall	Guest	10.10.0.192/26	0.0.0.0/0	DENY	guest anti-spoof printer

Billedet ovenfor viser de opsatte ACL'er.
1 prioritet er vores firewall ACL. Det er vigtigt at denne firewall regel er på 1 prioritet, grundet ACL'er bliver læst top -> down, så den første ACL der gælder, er den der bliver brugt. Vi vil gerne have at firewall ACL'en gælder over alt andet, derfor 1 prioritet.
Dernæst kommer ACL'erne for gæster, igen grundet top -> down læsning, er vores DENY ACL'er nød til at være af højere prioritet end vores allow til at gå på nettet. Dette gør at gæsterne kan gå på nettet, men kan ikke kommunikere med nogen af de andre subnets.
Employees er nødsaget til at have en specific ACL pr. Medarbejder som lukker for kommunikationen til gæster.
Admin/Employee subnettet har åbent for alt kommunikation.
Til sidst kommer der 2 firewall regler der stopper spoofing fra gæste subnettet til server og til printer subnetterne.

ICMP Bekræftelser

Vi testede alle 15 kommunikationsflows med 3 ICMP-pakker pr. flow, både før og efter vi satte ACL'erne op. Før ACL kunne alt pinge alt, også Guest til de interne netværk, så der var ingen segmentering og BR03 var brudt. Efter ACL opfører næsten alle flows sig, som kravene siger. Undtagelsen er Admin → Guest, som vi forklarer under tabellen.

Et ping-timeout kan have mange årsager, så det alene beviser ikke, at det er en ACL, der blokerer. Derfor aflæste vi hit-tællerne på reglerne før og efter testen. Når præcis den regel, der skulle blokere, tæller op med det forventede antal pakker, og ingen andre regler gør det, kan vi være sikre på årsagen.

#	Flow	BR	Forventet	Faktisk	Relevant Regel
1	Admin > Employee	BR01	ALLOW	OK	INGEN
2	Admin > Server	BR01	ALLOW	OK	ACL-7
3	Admin > Printer	BR01	ALLOW	OK	ACL-7
4	Admin > Guest	BR01	ALLOW	Request ud, reply droppet	Request: ACL-7. Reply: ACL-2
5	Admin > Internet	BR01	ALLOW	OK	ACL-7
6	Employee > Admin	BR02	ALLOW	OK	INGEN
7	Employee > Server	BR02	ALLOW	OK	ACL-7
8	Employee > Printer	BR02	ALLOW	OK	ACL-7
9	Employee > Internet	BR02	ALLOW	OK	ACL-7
10	Employee > Guest		DENY	Blockeret	ACL-6
11	Guest > Admin		DENY		ACL-2
12	Guest > Employee		DENY	Blockeret	ACL-2
13	Guest > Server		DENY	Blockeret	ACL-3
14	Guest > Printer		DENY	Blockeret	ACL-4
15	Guest > Internet		ALLOW	OK	ACL-5

Tabellen herunder viser, hvor meget hver regel steg under testen. Hver regel steg med præcis det antal pakker, vi havde forventet.

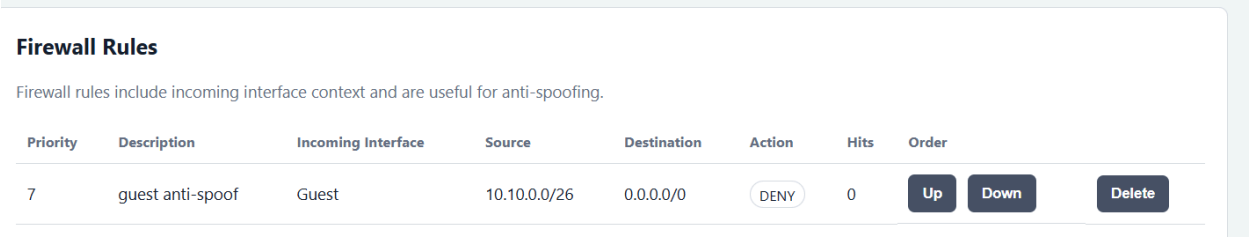
Regel	Før	Efter	Ændring	Forventet	Forklaring
ACL 2: guest to admin/employee (DENY)	21	30	+9	+9	Guest→Admin, Guest→Employee, Admin→Guest (reply)
ACL 3: guest to server (DENY)	7	10	+3	+3	Guest→Server
ACL 4: guest to printer (DENY)	7	10	+3	+3	Guest→Printer
ACL 5: guest to	11	14	+3	+3	Guest→Internet

internet (ALLOW)					
ACL 6: employee to guest (DENY)	11	14	+3	+3	Employee→Guest
ACL 7: admin + employee to all (ALLOW)	64	85	+21	+21	Admin→Server/Printer/Guest/Internet og Employee→Server/Printer/Internet

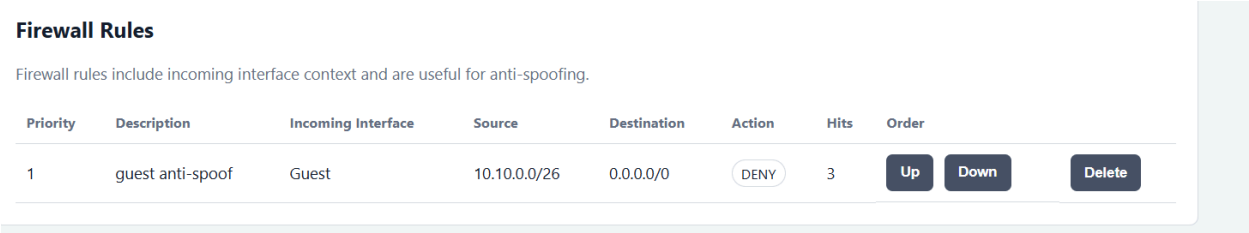
Der er én ting, man kan undre sig over: ACL 7 steg med 21 og ikke 27. Det er fordi Admin og Employee ligger på samme subnet (10.10.0.0/26). Trafik mellem dem går ikke igennem firewallen, så den rammer ingen ACL. De to flows (Admin ↔ Employee) kan derfor ikke dokumenteres med tællere *(Bilag A)*.

Spoofing analyse

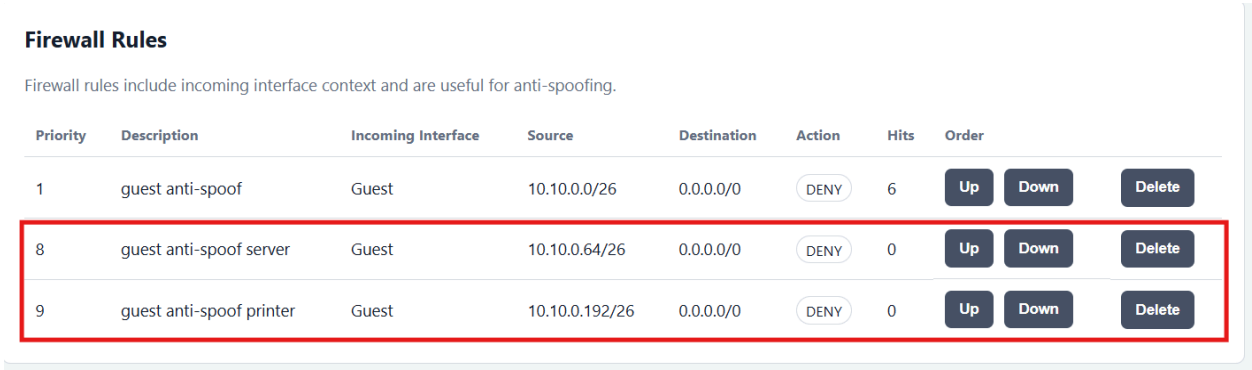
Før anti-spoofing: Fra Guest (10.10.0.140) sendte vi 3 ICMP-pakker med hping3 og Admins IP (10.10.0.10) som forfalsket afsender, mod Server (10.10.0.70). Tcpdump viste at Server modtog 3 ping requests fra 10.10.0.10, og svarede på dem, samt at Admin modtog 3 ping replies, som Admin aldrig havde bedt om. Ingen af Guest-DENY-reglerne blev ramt. Pakkerne matchede ACL-7 (source 10.10.0.0/26, ALLOW), fordi en ACL kun ser source og destinations IP og ikke, hvilket interface pakken kom ind på *(Bilag B)*.



Anti-spoofing og prioritet: Vi oprettede en firewall-regel (Incoming Zone: Guest, source 10.10.0.0/26, destination 0.0.0.0/0, DENY). Den blev først placeret på prioritet 7, efter ACL-7. Her virkede den ikke. Pakkerne nåede stadig frem, og reglen fik 0 hits. Første match vinder, så ACL-7 tillod pakken, før firewall-reglen blev evalueret. Efter flytning til prioritet 1 var tcpdump på Server og Admin tom, og reglen fik 3 hits, mens ACL-7 var uændret. Reglen skal stå øverst, fordi dens source overlapper med ACL-7 *(Bilag C)*.



Andre interne kilder: Spoofing af Server (10.10.0.70) og Printer (10.10.0.200) slap igennem, fordi de ligger uden for 10.10.0.0/26. Ingen regel matchede dem, de faldt på default ALLOW, og hit-tællerne var uændrede. Vi tilføjede derfor to firewall-regler (prioritet 8 og 9, source 10.10.0.64/26 og 10.10.0.192/26, DENY). Bagefter var tcpdump på Printer og Server tom, og reglerne steg med 3 hits hver. Ingen af ACL-reglerne matcher disse sources, så de to firewall-regler bliver evalueret uanset deres placering *(Bilag D)*.



Firewall Rules

Firewall rules include incoming interface context and are useful for anti-spoofing.

Priority	Description	Incoming Interface	Source	Destination	Action	Hits	Order		
1	guest anti-spoof	Guest	10.10.0.0/26	0.0.0.0/0	DENY	6	Up	Down	Delete
8	guest anti-spoof server	Guest	10.10.0.64/26	0.0.0.0/0	DENY	0	Up	Down	Delete
9	guest anti-spoof printer	Guest	10.10.0.192/26	0.0.0.0/0	DENY	0	Up	Down	Delete

8	guest anti-spoof server	Guest	10.10.0.64/26	0.0.0.0/0	DENY	3	Up	Down	Delete
9	guest anti-spoof printer	Guest	10.10.0.192/26	0.0.0.0/0	DENY	3	Up	Down	Delete

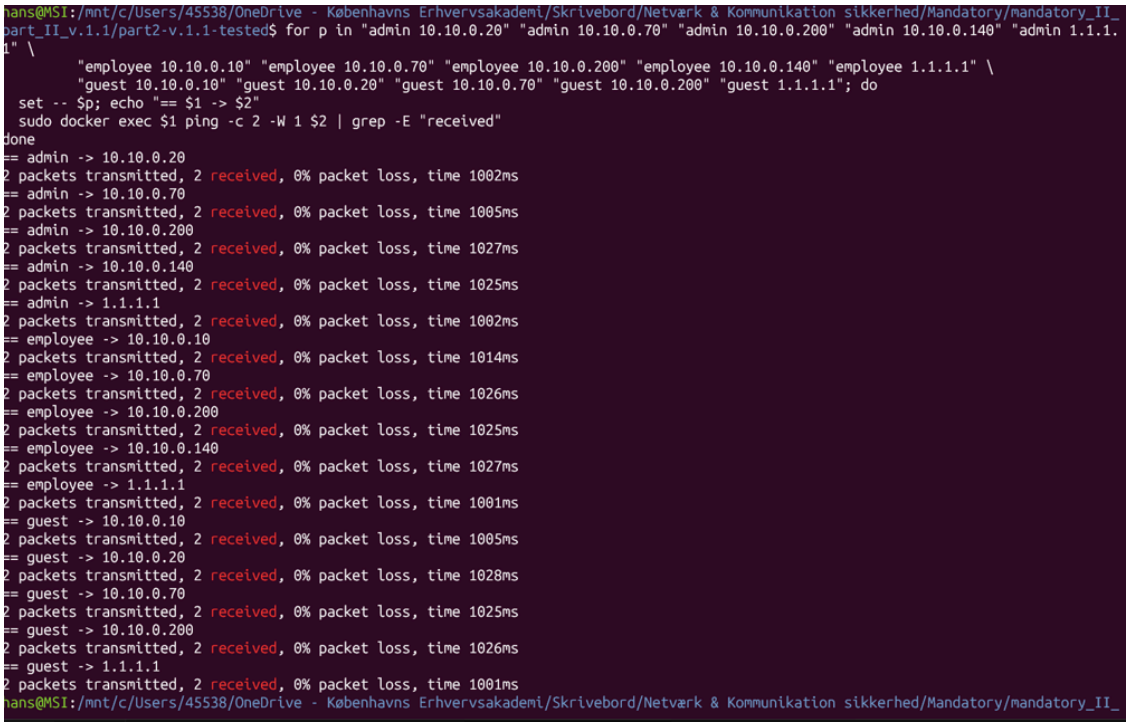
Konklusion

Overordnet set passer vores ACL opsætning med firewall til MoSec's requirements, men det er muligt at udvide yderligere med flere firewall regler mellem de forskellige subnets. Det ville yderlige segmentere netværkene og dermed også øge sikkerheden.
Der er blevet gjort overvejelser efterfølgende at det måske ville være smartere at opsætte Admin/Employee subnettet, så at der skal opsættes IP specifikke ACL'er for admins i stedet for employee's. Der er typisk flere medarbejdere end admins samt større udskiftning så det ville give mindre vedligeholdelse.

Bilag

Bilag A

Ping alle før ACL



Ping alle efter ACL

```
hans@MSI:/mnt/c/Users/45538/OneDrive - Københavns Erhvervsakademi/Skrivebord/Netværk & Kommunikation sikkerhed/Mandatory/mandatory_II_
part_II_v.1.1/part2-v.1.1-tested$ for p in "admin 10.10.0.70" "admin 10.10.0.200" "admin 1.1.1.1" \
        "employee 10.10.0.70" "employee 10.10.0.200" "employee 1.1.1.1" \
        "guest 10.10.0.10" "guest 10.10.0.20" "guest 10.10.0.70" "guest 10.10.0.200" "guest 1.1.1.1"; do
  set -- $p; echo "== $1 -> $2"
  sudo docker exec $1 ping -c 2 -W 1 $2 | grep -E "received"
done
== admin -> 10.10.0.70
[sudo: authenticate] Password:
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
== admin -> 10.10.0.200
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
== admin -> 1.1.1.1
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
== employee -> 10.10.0.70
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
== employee -> 10.10.0.200
2 packets transmitted, 2 received, 0% packet loss, time 1004ms
== employee -> 1.1.1.1
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
== guest -> 10.10.0.10
2 packets transmitted, 0 received, 100% packet loss, time 1021ms
== guest -> 10.10.0.20
2 packets transmitted, 0 received, 100% packet loss, time 1011ms
== guest -> 10.10.0.70
2 packets transmitted, 0 received, 100% packet loss, time 999ms
== guest -> 10.10.0.200
2 packets transmitted, 0 received, 100% packet loss, time 1007ms
== guest -> 1.1.1.1
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
hans@MSI:/mnt/c/Users/45538/OneDrive - Københavns Erhvervsakademi/Skrivebord/Netværk & Kommunikation sikkerhed/Mandatory/mandatory_II_
part_II_v.1.1/part2-v.1.1-tested$
```

```
part_II_v.1.1/part2-v.1.1-tested$ for p in "admin 10.10.0.140" "employee 10.10.0.140"; do
  set -- $p; echo "== $1 -> $2"
  sudo docker exec $1 ping -c 2 -W 1 $2 | grep -E "received"
done
== admin -> 10.10.0.140
2 packets transmitted, 0 received, 100% packet loss, time 1014ms
== employee -> 10.10.0.140
2 packets transmitted, 0 received, 100% packet loss, time 1010ms
```

Bilag B

Pinger fra Guest men lades som jeg er Admin - pinger til server: FØR FIREWALL

```
hans@MSI:~$ sudo docker exec guest hping3 --icmp -c 3 -a 10.10.0.10 10.10.0.70

--- 10.10.0.70 hping statistic ---
3 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
HPING 10.10.0.70 (eth0 10.10.0.70): icmp mode set, 28 headers + 0 data bytes
hans@MSI:~$
```

```
hans@MSI:~$ sudo docker exec admin tcpdump -nli any icmp
tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
12:11:12.640621 eth0  In  IP 10.10.0.70 > 10.10.0.10: ICMP echo reply, id 29440, seq 0, length 8
12:11:13.640893 eth0  In  IP 10.10.0.70 > 10.10.0.10: ICMP echo reply, id 29440, seq 256, length 8
12:11:14.641068 eth0  In  IP 10.10.0.70 > 10.10.0.10: ICMP echo reply, id 29440, seq 512, length 8
```

```
hans@MSI:~$ sudo docker exec server tcpdump -nli any icmp
tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
12:11:12.640521 eth0  In  IP 10.10.0.10 > 10.10.0.70: ICMP echo request, id 29440, seq 0, length 8
12:11:12.640591 eth0  Out IP 10.10.0.70 > 10.10.0.10: ICMP echo reply, id 29440, seq 0, length 8
12:11:13.640831 eth0  In  IP 10.10.0.10 > 10.10.0.70: ICMP echo request, id 29440, seq 256, length 8
12:11:13.640863 eth0  Out IP 10.10.0.70 > 10.10.0.10: ICMP echo reply, id 29440, seq 256, length 8
12:11:14.641031 eth0  In  IP 10.10.0.10 > 10.10.0.70: ICMP echo request, id 29440, seq 512, length 8
12:11:14.641049 eth0  Out IP 10.10.0.70 > 10.10.0.10: ICMP echo reply, id 29440, seq 512, length 8
```

Bilag C

Pinger fra Guest men lades som jeg er Admin - pinger til server - EFTER FIREWALL prioritet

```
hans@MSI:~$ sudo docker exec guest hping3 --icmp -c 3 -a 10.10.0.10 10.10.0.70
[sudo: authenticate] Password:
HPING 10.10.0.70 (eth0 10.10.0.70): icmp mode set, 28 headers + 0 data bytes

--- 10.10.0.70 hping statistic ---
3 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
hans@MSI:~$
```

```
hans@MSI:~$ sudo docker exec admin tcpdump -nli any icmp
[sudo: authenticate] Password:
tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
```

```
hans@MSI:~$ sudo docker exec server tcpdump -nli any icmp
[sudo: authenticate] Password:
tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
```

Bilag D

Pinger fra Guest men lades som jeg er server - pinger til printer - FØR FIREWALL

```
hans@MSI:~$ sudo docker exec guest hping3 --icmp -c 3 -a 10.10.0.70 10.10.0.200
[sudo: authenticate] Password:

--- 10.10.0.200 hping statistic ---
3 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
HPING 10.10.0.200 (eth0 10.10.0.200): icmp mode set, 28 headers + 0 data bytes
hans@MSI:~$
```

```
hans@MSI:~$ sudo docker exec printer tcpdump -nli any icmp
[sudo: authenticate] Password:
tcpdump: WARNING: any: That device doesn't support promiscuous mode
(Promiscuous mode not supported on the "any" device)
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
10:49:52.187829 eth0  In  IP 10.10.0.70 > 10.10.0.200: ICMP echo request, id 45824, seq 0, length 8
10:49:52.197871 eth0  Out IP 10.10.0.200 > 10.10.0.70: ICMP echo reply, id 45824, seq 0, length 8
10:49:53.201743 eth0  In  IP 10.10.0.70 > 10.10.0.200: ICMP echo request, id 45824, seq 256, length 8
10:49:53.201770 eth0  Out IP 10.10.0.200 > 10.10.0.70: ICMP echo reply, id 45824, seq 256, length 8
10:49:54.203355 eth0  In  IP 10.10.0.70 > 10.10.0.200: ICMP echo request, id 45824, seq 512, length 8
10:49:54.203383 eth0  Out IP 10.10.0.200 > 10.10.0.70: ICMP echo reply, id 45824, seq 512, length 8
10:52:41.594275 eth0  In  IP 10.10.0.70 > 10.10.0.200: ICMP echo request, id 47360, seq 0, length 8
10:52:41.594343 eth0  Out IP 10.10.0.200 > 10.10.0.70: ICMP echo reply, id 47360, seq 0, length 8
10:52:42.595990 eth0  In  IP 10.10.0.70 > 10.10.0.200: ICMP echo request, id 47360, seq 256, length 8
10:52:42.596019 eth0  Out IP 10.10.0.200 > 10.10.0.70: ICMP echo reply, id 47360, seq 256, length 8
10:52:43.597114 eth0  In  IP 10.10.0.70 > 10.10.0.200: ICMP echo request, id 47360, seq 512, length 8
10:52:43.597146 eth0  Out IP 10.10.0.200 > 10.10.0.70: ICMP echo reply, id 47360, seq 512, length 8
```

Pinger fra Guest men lades som jeg er printer - pinger til server - FØR FIREWALL

```
hans@MSI:~$ sudo docker exec guest hping3 --icmp -c 3 -a 10.10.0.200 10.10.0.70

--- 10.10.0.70 hping statistic ---
3 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
HPING 10.10.0.70 (eth0 10.10.0.70): icmp mode set, 28 headers + 0 data bytes
hans@MSI:~$
```

```
hans@MSI:~$ sudo docker exec server tcpdump -nli any icmp
tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
10:55:05.558308 eth0  In  IP 10.10.0.200 > 10.10.0.70: ICMP echo request, id 49152, seq 0, length 8
10:55:05.559242 eth0  Out IP 10.10.0.70 > 10.10.0.200: ICMP echo reply, id 49152, seq 0, length 8
10:55:06.559706 eth0  In  IP 10.10.0.200 > 10.10.0.70: ICMP echo request, id 49152, seq 256, length 8
10:55:06.559785 eth0  Out IP 10.10.0.70 > 10.10.0.200: ICMP echo reply, id 49152, seq 256, length 8
10:55:07.560131 eth0  In  IP 10.10.0.200 > 10.10.0.70: ICMP echo request, id 49152, seq 512, length 8
10:55:07.560182 eth0  Out IP 10.10.0.70 > 10.10.0.200: ICMP echo reply, id 49152, seq 512, length 8
```

Pinger fra Guest men lades som jeg er server - pinger til printer - EFTER FIREWALL

```
hans@MSI:~$    sudo docker exec guest hping3 --icmp -c 3 -a 10.10.0.70 10.10.0.200

--- 10.10.0.200 hping statistic ---
3 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
HPING 10.10.0.200 (eth0 10.10.0.200): icmp mode set, 28 headers + 0 data bytes
```

```
hans@MSI:~$ sudo docker exec printer tcpdump -nli any icmp
tcpdump: WARNING: any: That device doesn't support promiscuous mode
(Promiscuous mode not supported on the "any" device)
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
```

Pinger fra Guest men lades som jeg er printer - pinger til server - EFTER FIREWALL

```
hans@MSI:~$    sudo docker exec guest hping3 --icmp -c 3 -a 10.10.0.200 10.10.0.70
HPING 10.10.0.70 (eth0 10.10.0.70): icmp mode set, 28 headers + 0 data bytes

--- 10.10.0.70 hping statistic ---
3 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

```
hans@MSI:~$ sudo docker exec server tcpdump -nli any icmp
tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
```